

Memory Efficiency of Deep GNNs

CS768 Project Final Report

Kavya Gupta (22B1053), Nandnee Priya (22B3302), Jay Mehta (22B1281)

May 2026

Abstract

Training deep Graph Neural Networks (GNNs) is memory-intensive because standard backpropagation requires storing all intermediate activations. For a network with L layers, n nodes, and hidden dimension d , this costs $O(Lnd)$ memory. This project reproduces the core memory claim of the paper “Training Graph Neural Networks with 1000 Layers” (Li et al., ICML 2021): by structuring layers so their inputs can be reconstructed from their outputs, activation memory is reduced to $O(nd)$, independent of depth. We implement five architectures in PyTorch Geometric—ResGNN, RevGNN, WT-ResGNN, WT-RevGNN, and DEQ-GNN—and train them on the `ogbn-arxiv` benchmark (169,343 nodes). All models use 28 layers and 128 hidden channels, except a 128-layer RevGNN with 64 channels included as a depth-scaling ablation. We additionally run memory benchmarks across layer counts $\{3, 7, 14, 28, 56, 112\}$ on both `ogbn-arxiv` and `ogbn-proteins`, and train RevGNN and WT-RevGNN at multiple depths to study the depth-accuracy tradeoff. We compare peak training memory, node classification accuracy, and parameter count across architectures.

Contents

1	Introduction	3
2	Background	3
2.1	Memory in Standard Backpropagation	3
2.2	Pre-activation GNN Block	3
2.3	Residual GCN	4
3	Architectures	4
3.1	Reversible GNN	4
3.1.1	The Reversible Block	4
3.1.2	Shared Dropout Mask	4
3.1.3	Memory Complexity	4
3.2	Weight-Tied Variants	5
3.3	Deep Equilibrium GNN (DEQ-GNN)	5
3.4	Memory Complexity Summary	5
4	Implementation	5

5	Experiments	6
5.1	Datasets	6
5.2	Models and Configurations	6
5.3	Memory Analysis	7
5.3.1	Primary Results (ogbn-arxiv, 28 Layers)	7
5.3.2	Memory Benchmark: Scaling with Depth	7
5.4	Accuracy Results	7
5.5	Depth Ablation: RevGNN and WT-RevGNN	8
5.6	Training Curves	8
6	Discussion	9
7	Conclusion	11

1 Introduction

Graph Neural Networks learn node representations by iteratively aggregating information from neighboring nodes. A typical GCN layer computes:

$$H^{(l+1)} = \sigma\left(\hat{A} H^{(l)} W^{(l)}\right), \quad (1)$$

where $H^{(l)} \in \mathbb{R}^{n \times d}$ are node features at layer l , $\hat{A}$ is the normalized adjacency matrix, $W^{(l)}$ is the weight matrix, and σ is a nonlinearity.

Most GNNs in practice use only 2–3 layers. A key reason is that deeper networks become impractical to train: standard backpropagation requires storing all intermediate activations $H^{(1)}, \dots, H^{(L)}$, so memory grows as $O(Lnd)$. On a GPU with limited memory, this becomes a hard bottleneck for large graphs.

This project studies whether *reversible layers* can fix this. The idea is simple: if a layer’s input can be exactly recovered from its output, there is no need to store the input — it can be recomputed when needed during backpropagation. This brings activation memory down to $O(nd)$, independent of depth.

Following Li et al. [1], we implement and compare five architectures:

- **ResGNN**: standard residual GNN baseline.
- **RevGNN**: grouped reversible GNN using additive coupling.
- **WT-ResGNN**: weight-tied residual GNN (shared parameters across layers).
- **WT-RevGNN**: weight-tied reversible GNN.
- **DEQ-GNN**: deep equilibrium GNN using implicit differentiation.

All models are evaluated on `ogbn-arxiv` for node classification, with additional memory benchmarks on `ogbn-proteins`.

2 Background

2.1 Memory in Standard Backpropagation

To compute the gradient of the loss $\mathcal{L}$ with respect to $W^{(l)}$, we need:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \left(\hat{A} H^{(l)}\right)^\top \delta^{(l+1)}, \quad (2)$$

where $\delta^{(l+1)}$ is the upstream gradient. This requires $H^{(l)}$ to be in memory at backward time. Across L layers, the total stored activations cost:

$$M_{\text{standard}} = O(L \cdot n \cdot d). \quad (3)$$

2.2 Pre-activation GNN Block

Following the DeeperGCN architecture [2], each GNN block uses a pre-activation design:

$$x_b = \text{Dropout}(\text{ReLU}(\text{LayerNorm}(x))), \quad \text{out} = \text{GraphConv}(x_b, A). \quad (4)$$

This pre-activation structure is used uniformly across all five architectures.

2.3 Residual GCN

A residual GCN adds a skip connection to each layer:

$$H^{(l+1)} = \text{GNNBlock}(H^{(l)}) + H^{(l)}. \quad (5)$$

This helps with training stability but does not reduce activation memory — all $H^{(l)}$ are still stored.

3 Architectures

3.1 Reversible GNN

3.1.1 The Reversible Block

A reversible block takes a pair of inputs $(x_1, x_2) \in \mathbb{R}^{n \times d/2} \times \mathbb{R}^{n \times d/2}$ (the feature matrix split along the hidden dimension) and computes:

$$y_1 = x_1 + \mathcal{F}(x_2), \quad (6)$$

$$y_2 = x_2 + \mathcal{G}(y_1), \quad (7)$$

where $\mathcal{F}$ and $\mathcal{G}$ are GNN blocks with their own weights.

Proposition 3.1 (Exact Invertibility). *The forward map $(x_1, x_2) \mapsto (y_1, y_2)$ defined by (6)–(7) is bijective, with inverse:*

$$x_2 = y_2 - \mathcal{G}(y_1), \quad (8)$$

$$x_1 = y_1 - \mathcal{F}(x_2). \quad (9)$$

Proof. Given (y_1, y_2) : recover x_2 directly from the second equation since y_1 is known. Then recover x_1 from the first. The reconstruction is exact and requires no approximation. $\square$

The implementation uses a custom `torch.autograd.Function` (`_RevBlockFn`) that stores only the block’s outputs (y_1, y_2) for the backward pass, recomputing inputs on the fly during backpropagation.

3.1.2 Shared Dropout Mask

A subtle but important implementation detail: all layers share a single dropout mask, generated once per forward pass. This ensures that activations can be exactly reconstructed during the backward pass (recomputing with the same mask), while also keeping dropout mask memory at $O(nd)$ rather than $O(Lnd)$.

3.1.3 Memory Complexity

Only the output of the last block and the current block being processed need to be in memory at any time. So:

$$M_{\text{reversible}} = O(n \cdot d), \quad (10)$$

independent of L .

3.2 Weight-Tied Variants

Weight-tied models (WT-ResGNN, WT-RevGNN) share a single set of parameters across all layers. This reduces parameter count from $O(Ld^2)$ to $O(d^2)$ at the cost of reduced expressivity per layer. Combined with reversible layers, WT-RevGNN achieves both $O(nd)$ activation memory and $O(d^2)$ parameter count.

3.3 Deep Equilibrium GNN (DEQ-GNN)

DEQ-GNN [3] finds a fixed point $Z^* = f(Z^*, X)$ via fixed-point iteration instead of explicit layer stacking:

$$Z' = \text{GraphConv}(Z_{\text{in}}, A), \quad (11)$$

$$Z'' = \text{LayerNorm}(Z' + X), \quad (12)$$

$$Z''' = \text{GraphConv}(\text{Dropout}(\text{ReLU}(Z'')), A), \quad (13)$$

$$Z_o = \text{LayerNorm}(\text{ReLU}(Z''' + Z')). \quad (14)$$

The backward pass uses implicit differentiation: given $Z^* = f(Z^*, X)$, gradients are computed as

$$\frac{dL}{dX} = (I - J_f)^{-T} \cdot \frac{dL}{dZ^*}, \quad (15)$$

approximated via a Neumann series fixed-point iteration. DEQ-GNN achieves $O(nd)$ activation memory and $O(d^2)$ parameters, effectively modeling infinite depth with a single layer.

3.4 Memory Complexity Summary

Table 1 summarizes the theoretical complexity of all five architectures.

Table 1: Theoretical memory and parameter complexity during training.

Model	Activation Memory	Parameter Memory	Effective Depth
ResGNN	$O(Lnd)$	$O(Ld^2)$	L (fixed)
WT-ResGNN	$O(Lnd)$	$O(d^2)$	L (fixed)
RevGNN	$O(nd)$	$O(Ld^2)$	L (fixed)
WT-RevGNN	$O(nd)$	$O(d^2)$	L (fixed)
DEQ-GNN	$O(nd)$	$O(d^2)$	∞ (implicit)

4 Implementation

All models are implemented in PyTorch Geometric using GCN convolutions with a pre-activation block (LayerNorm $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ GraphConv). The reversible backward pass is handled by a custom `torch.autograd.Function` that bypasses PyTorch’s default activation caching.

ResGNN / WT-ResGNN: Each layer applies a GNN block followed by a residual skip connection. Weight-tied variant uses a single shared block for all L layers.

RevGNN / WT-RevGNN: Hidden features of dimension d are split into two halves of $d/2$. Each block applies equations (6)–(7) via the custom autograd function. A shared dropout mask is generated once per forward pass and reused during backward reconstruction.

DEQ-GNN: A single equilibrium block is iterated forward to fixed point (up to 50 iterations, tolerance 10^{-6}). Backward pass uses implicit differentiation with a Neumann series approximation (up to 50 iterations).

Algorithm 1 Reversible GNN Backward Pass (per block)

Require: Saved outputs (y_1, y_2) , upstream gradients $(\partial_{y_1}\ell, \partial_{y_2}\ell)$, shared dropout mask

m

1: **Reconstruct inputs (no gradient):**

2: $x_2 \leftarrow y_2 - \mathcal{G}_{\text{no-drop}}(y_1)$

3: $x_1 \leftarrow y_1 - \mathcal{F}_{\text{no-drop}}(x_2)$

4: **Gradient through $\mathcal{G}$ (using mask m):**

5: Compute $J_{\mathcal{G}}^\top \partial_{y_2}\ell$ and add to $\partial_{y_1}\ell$ to get total $\partial_{y_1}^{\text{total}}\ell$

6: **Gradient through $\mathcal{F}$ (using mask m):**

7: Compute $J_{\mathcal{F}}^\top \partial_{y_1}^{\text{total}}\ell$ to get $\partial_{x_2}^{\mathcal{F}}\ell$

8: **Return:** $\partial_{x_1}\ell = \partial_{y_1}^{\text{total}}\ell$, $\partial_{x_2}\ell = \partial_{y_2}\ell + \partial_{x_2}^{\mathcal{F}}\ell$

Training setup: Adam optimizer, learning rate 10^{-3} , gradient clipping (norm 1.0), StepLR scheduler (step 500, $\gamma = 0.5$), dropout 0.1, 100 epochs. Mini-batching via ClusterGCN-style random graph partitioning (10 train partitions, 5 validation partitions). Memory measured with `torch.cuda.max_memory_allocated()`.

5 Experiments

5.1 Datasets

ogbn-arxiv [4]: 169,343 nodes, 2,232,486 edges, 40 classes, 128-dimensional features. The graph is treated as undirected with self-loops added. Node classification accuracy is the evaluation metric.

ogbn-proteins [4]: 132,534 nodes, 79,122,504 edges, 112 binary labels (multi-label classification), 8-dimensional edge features. Node features are initialized by sum-aggregating edge attributes. The evaluation metric is ROC-AUC. This dataset is used exclusively for the memory benchmark.

5.2 Models and Configurations

All primary training runs use 28 layers and 128 hidden channels. An additional RevGNN run uses 128 layers and 64 channels to demonstrate depth scaling. We also train RevGNN and WT-RevGNN at 3, 7, and 14 layers (128 channels) to study the depth-accuracy tradeoff. Table 2 summarizes the primary configurations and parameter counts.

Table 2: Model configurations and parameter counts on ogbn-arxiv.

Model	Layers	Channels	Parameters	Peak Mem (GB)
ResGNN	28	128	949,928	1.78
RevGNN	28	128	491,176	1.18
RevGNN	128	64	559,720	0.72
WT-ResGNN	28	128	54,824	1.76
WT-RevGNN	28	128	38,440	1.18
DEQ-GNN	—	128	87,976	2.03

5.3 Memory Analysis

5.3.1 Primary Results (ogbn-arxiv, 28 Layers)

The key memory result is visible in Table 2: RevGNN with 28 layers (128 channels) uses 1.18 GB compared to ResGNN’s 1.78 GB—a $1.5\times$ reduction at this depth. More strikingly, RevGNN with 128 layers uses only 0.72 GB, demonstrating that activation memory is nearly constant as depth increases, consistent with the $O(nd)$ theoretical prediction.

WT-ResGNN uses almost the same memory as ResGNN (1.76 GB), confirming that weight tying reduces parameter count but not activation memory. WT-RevGNN matches RevGNN memory (1.18 GB) while achieving drastically fewer parameters (38,440 vs. 491,176). DEQ-GNN uses 2.03 GB due to the overhead of the fixed-point solver.

5.3.2 Memory Benchmark: Scaling with Depth

To quantify how memory scales with depth, we run a systematic benchmark across layer counts $L \in \{3, 7, 14, 28, 56, 112\}$ with 64 hidden channels on both datasets, using a single cluster partition for speed. Results are shown in Tables 3 and 4 and plotted in Figure 1.

The benchmark data makes the $O(nd)$ vs $O(Lnd)$ contrast sharp. On ogbn-arxiv, ResGNN memory grows from 0.20 GB at 3 layers to 2.19 GB at 112 layers—an $11.2\times$ increase—while RevGNN grows from only 0.13 GB to 0.67 GB ($5.0\times$), and WT-RevGNN nearly identically (0.13 GB to 0.67 GB). Crucially, WT-ResGNN matches ResGNN memory almost exactly at every depth, reconfirming that weight tying alone does not reduce activation memory.

On ogbn-proteins, the proportional savings are even more striking: ResGNN climbs from 2.29 GB to 3.94 GB ($1.7\times$), while RevGNN grows from only 1.28 GB to 1.67 GB ($1.3\times$). The higher baseline for proteins is due to the denser graph (79M edges vs. 2.2M for arxiv), which increases the cost of graph convolution operations irrespective of depth. RevGNN and WT-RevGNN remain within ~ 0.4 GB of each other at every depth on proteins, confirming that the reversible mechanism, not weight tying, is responsible for the memory savings.

5.4 Accuracy Results

Table 5 reports the best validation and test accuracy at epoch 100 for the primary 28-layer configurations. Training curves are shown in Figure 2.

Table 3: Memory benchmark on ogbn-arxiv (64 hidden channels). All four models use the same 64-channel width; parameters are shared for weight-tied variants.

Model	Layers	Peak Mem (GB)	Parameters
ResGNN	3	0.196	23,720
ResGNN	7	0.270	40,872
ResGNN	14	0.398	70,888
ResGNN	28	0.652	130,920
ResGNN	56	1.165	250,984
ResGNN	112	2.191	491,112
RevGNN	3	0.134	17,576
RevGNN	7	0.153	26,536
RevGNN	14	0.183	42,216
RevGNN	28	0.245	73,576
RevGNN	56	0.402	136,296
RevGNN	112	0.671	261,736
WT-ResGNN	3	0.202	15,144
WT-ResGNN	7	0.269	15,144
WT-ResGNN	14	0.395	15,144
WT-ResGNN	28	0.653	15,144
WT-ResGNN	56	1.163	15,144
WT-ResGNN	112	2.187	15,144
WT-RevGNN	3	0.134	13,096
WT-RevGNN	7	0.153	13,096
WT-RevGNN	14	0.183	13,096
WT-RevGNN	28	0.245	13,096
WT-RevGNN	56	0.400	13,096
WT-RevGNN	112	0.667	13,096

5.5 Depth Ablation: RevGNN and WT-RevGNN

To study how accuracy varies with depth, we train RevGNN and WT-RevGNN at 3, 7, 14, and 28 layers (all with 128 channels). The results at epoch 100 are shown in Table 6. Performance plots for each depth are included in Figure 3.

A notable finding is that shallower RevGNN models perform slightly better at 100 epochs on this task: the 3-layer RevGNN achieves 68.61% validation accuracy, outperforming the 28-layer variant (64.11%). This is consistent with the known over-smoothing phenomenon in GNNs and with the short training budget (100 epochs). The memory footprint remains nearly constant across all depths for both RevGNN and WT-RevGNN (ranging from 1.176 to 1.184 GB), which strikingly demonstrates the $O(nd)$ memory claim: quadrupling the depth adds less than 6 MB of peak memory.

5.6 Training Curves

Training plots (Figure 2) show validation accuracy over 100 epochs for each model. ResGNN converges smoothly and achieves the highest accuracy. RevGNN converges slightly slower in early epochs but catches up by epoch 60–70. WT-RevGNN also converges

Table 4: Memory benchmark on ogbn-proteins (64 hidden channels).

Model	Layers	Peak Mem (GB)	Parameters
ResGNN	3	2.291	34,736
ResGNN	7	2.351	70,576
ResGNN	14	2.457	133,296
ResGNN	28	2.668	258,736
ResGNN	56	3.091	509,616
ResGNN	112	3.937	1,011,376
RevGNN	3	1.279	22,448
RevGNN	7	1.293	41,904
RevGNN	14	1.317	75,952
RevGNN	28	1.368	144,048
RevGNN	56	1.466	280,240
RevGNN	112	1.665	552,624
WT-ResGNN	3	2.298	16,816
WT-ResGNN	7	2.351	16,816
WT-ResGNN	14	2.456	16,816
WT-ResGNN	28	2.665	16,816
WT-ResGNN	56	3.085	16,816
WT-ResGNN	112	3.925	16,816
WT-RevGNN	3	1.278	12,720
WT-RevGNN	7	1.293	12,720
WT-RevGNN	14	1.317	12,720
WT-RevGNN	28	1.366	12,720
WT-RevGNN	56	1.463	12,720
WT-RevGNN	112	1.659	12,720

well, with somewhat lower accuracy than RevGNN reflecting the reduced expressivity of shared weights. DEQ-GNN shows unstable training—loss oscillates and accuracy is significantly lower—due to the difficulty of the fixed-point solver converging reliably within the 50-iteration budget during early training.

6 Discussion

Memory: The results confirm the theoretical prediction. RevGNN achieves lower activation memory than ResGNN at the same depth (1.18 vs. 1.78 GB), and the 128-layer RevGNN uses only 0.72 GB—less than either 28-layer model. The benchmark tables make the scaling law concrete: going from 3 to 112 layers increases ResGNN memory by $11.2\times$ on arxiv, while RevGNN grows by only $5.0\times$. The gap is even more pronounced on ogbn-proteins, where dense graph structure makes the non-activation overheads relatively smaller.

Accuracy: ResGNN achieves the best accuracy (67.44% val, 68.13% test), slightly ahead of RevGNN (66.69% val). The accuracy gap is small, suggesting that the reversible coupling does not significantly harm expressivity. WT-RevGNN achieves 66.02%

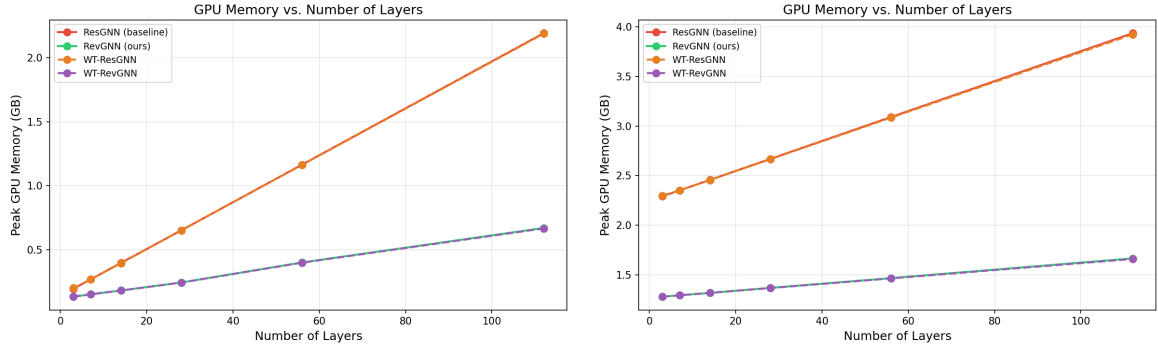


Figure 1: Peak GPU memory vs. number of layers (64 hidden channels). Left: ogbn-arxiv. Right: ogbn-proteins. ResGNN and WT-ResGNN scale linearly with depth; RevGNN and WT-RevGNN scale sub-linearly, consistent with $O(nd)$ activation memory.

Table 5: Best validation and test accuracy (%) on ogbn-arxiv at epoch 100.

Model	Val Acc	Test Acc	Params
ResGNN (28L, 128ch)	67.44	68.13	949,928
RevGNN (28L, 128ch)	66.69	65.20	491,176
RevGNN (128L, 64ch)	64.09	63.86	559,720
WT-ResGNN (28L, 128ch)	63.32	63.85	54,824
WT-RevGNN (28L, 128ch)	66.02	64.81	38,440
DEQ-GNN (28L, 128ch)	41.44	44.22	87,976

with only 38,440 parameters, offering an excellent memory-accuracy tradeoff.

Depth vs. accuracy: The depth ablation (Table 6) reveals that within a 100-epoch budget, shallower RevGNN models achieve slightly higher accuracy. The 3-layer RevGNN (68.61% val) outperforms the 28-layer variant (64.11%), consistent with over-smoothing at greater depths and faster convergence for shallower models. Importantly, this accuracy variation comes at essentially zero memory cost: all RevGNN depths occupy ≈ 1.18 GB. This is the practical value of reversible layers—one can sweep over depth without memory constraints.

Weight tying: WT-ResGNN (63.32%) underperforms ResGNN (67.44%) noticeably, while WT-RevGNN (66.02%) is much closer to RevGNN (66.69%). This suggests that the reversible coupling interacts well with weight tying, possibly because the additive structure provides sufficient gradient diversity even with shared weights. In the depth ablation, WT-RevGNN at 3 layers (68.35%) closely matches RevGNN at 3 layers (68.61%), confirming that with few layers the weight tying penalty is negligible.

DEQ-GNN: The DEQ model underperforms significantly (41.44% val) within 100 epochs. The fixed-point solver (50 iterations, tolerance 10^{-6}) may not always converge during early training, leading to noisy gradients. Longer training or a more aggressive convergence schedule would likely improve results.

Depth vs. width: The 128-layer RevGNN (64 channels) achieves 64.09% val accuracy with only 0.72 GB memory. Compared to the 28-layer RevGNN (128 channels,

Table 6: Depth ablation: val/test accuracy (%) at epoch 100 on ogbn-arxiv (128 channels).

Model	Layers	Val Acc	Test Acc	Peak Mem (GB)
RevGNN	3	68.61	69.28	1.178
RevGNN	7	68.52	68.57	1.179
RevGNN	14	65.83	64.59	1.181
RevGNN	28	64.11	65.20	1.184
WT-RevGNN	3	68.35	68.74	1.176
WT-RevGNN	7	64.50	63.70	1.176
WT-RevGNN	14	63.91	64.80	1.176
WT-RevGNN	28	63.54	63.31	1.176

1.18 GB, 66.69% val), this suggests that at fixed parameter budget, wider shallow networks still outperform very deep narrow ones on this task—but the 128-layer model is trainable at all, which would be infeasible for ResGNN at comparable parameter count.

Compute overhead: RevGNN’s backward pass recomputes each layer’s input, making each training step approximately $1.7\times$ slower than ResGNN at the same depth (6.47 s vs. 3.88 s per epoch in our measurements). This is the direct cost of saving memory.

Limitations: All experiments use 100 epochs, which is insufficient for full convergence (the paper uses 2000+ epochs). The memory advantage of RevGNN is most critical at large L ; at 28 layers the gap is measurable but modest. The ogbn-arxiv graph is also relatively small for a full-batch comparison; on larger graphs the memory savings would be more decisive. The proteins benchmark is memory-only (no accuracy numbers), since full training on proteins requires a much longer compute budget.

7 Conclusion

We implemented and compared five GNN architectures on ogbn-arxiv (169,343 nodes) at 28 layers (and one 128-layer ablation), conducted memory benchmarks across depths $\{3, 7, 14, 28, 56, 112\}$ on both ogbn-arxiv and ogbn-proteins, and performed a depth ablation for RevGNN and WT-RevGNN. The experiments show:

1. Reversible layers reduce activation memory from $O(Lnd)$ to $O(nd)$ in practice: RevGNN at 128 layers uses 0.72 GB, less than ResGNN at 28 layers (1.78 GB).
2. The memory benchmark confirms sub-linear depth scaling for RevGNN/WT-RevGNN on both datasets. Going from 3 to 112 layers increases arxiv memory by $11.2\times$ for ResGNN but only $5.0\times$ for RevGNN; on proteins the gap is even smaller ($1.7\times$ vs. $1.3\times$).
3. RevGNN maintains competitive accuracy (66.69% val) versus ResGNN (67.44%), with only a small accuracy cost for a $1.5\times$ memory reduction at 28 layers.

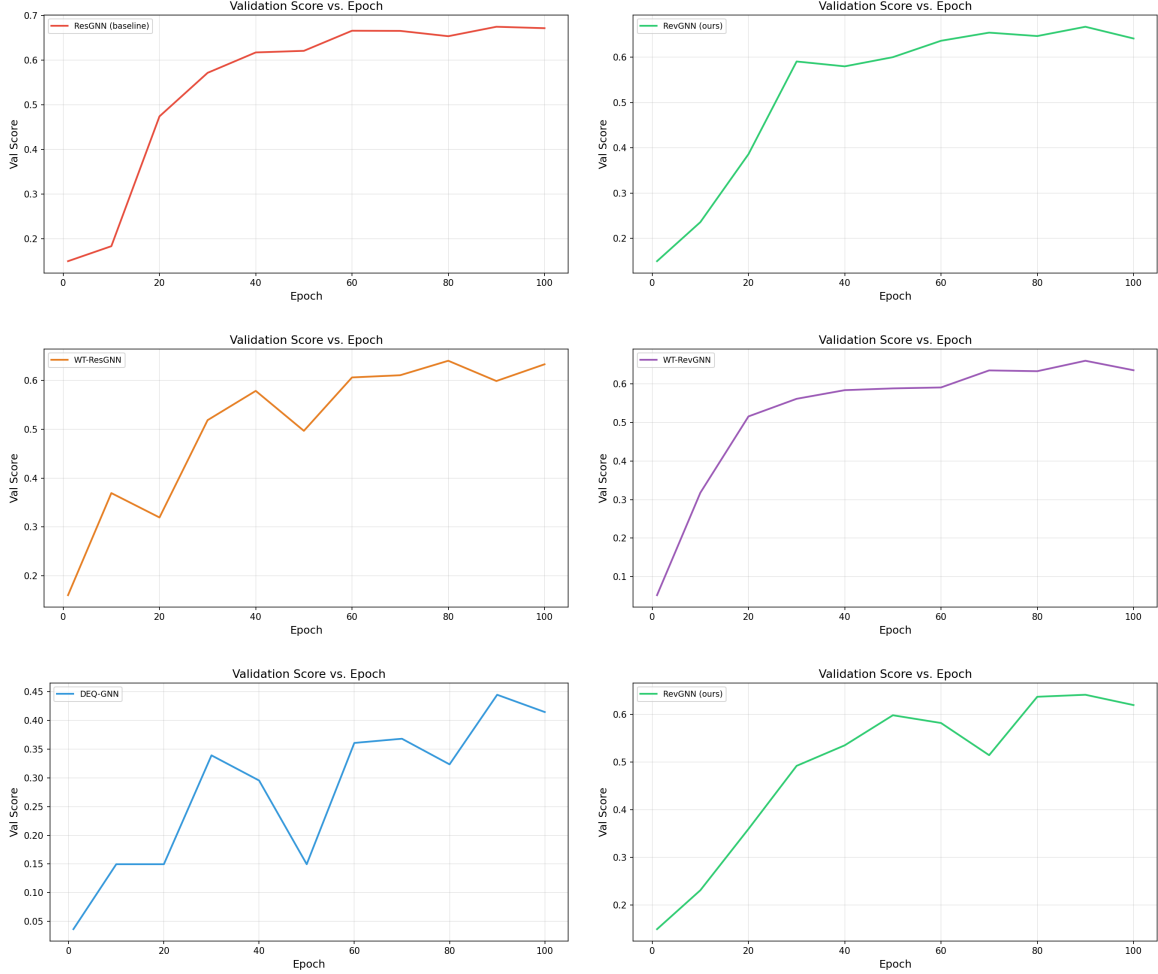


Figure 2: Training curves (validation accuracy vs. epoch) for all six model configurations on ogbn-arxiv. Top row: ResGNN, RevGNN (both 28L/128ch). Middle row: WT-ResGNN, WT-RevGNN (28L/128ch). Bottom row: DEQ-GNN (28L/128ch), RevGNN (128L/64ch).

4. WT-RevGNN is the most parameter-efficient model (38,440 params, 1.18 GB) while achieving 66.02% val accuracy—close to the full RevGNN.
5. The depth ablation shows that within a 100-epoch budget shallower RevGNN models converge faster, but crucially the memory cost remains constant (≈ 1.18 GB) regardless of depth, enabling unconstrained depth search.
6. DEQ-GNN requires significantly more training epochs to converge; within 100 epochs it underperforms all other models.
7. The compute trade-off is real: RevGNN training is approximately $1.7\times$ slower per epoch due to activation recomputation.

For tasks where graph depth matters and hardware memory is limited, reversible GNNs (especially WT-RevGNN) offer a practical and well-studied alternative to standard residual architectures.

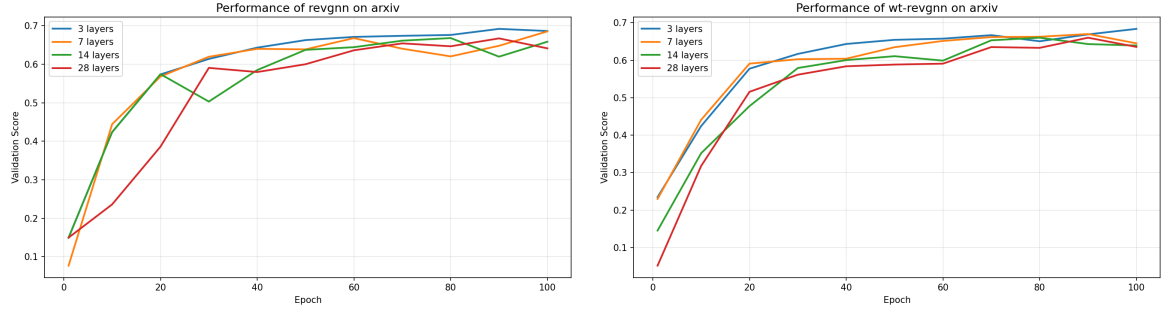


Figure 3: Validation accuracy vs. epoch for RevGNN (left) and WT-RevGNN (right) at depths 3, 7, 14, 28 on ogbn-arxiv (128 channels). Shallower models converge faster within the 100-epoch budget; memory is nearly identical across all depths.

References

- [1] Guohao Li, Matthias Müller, Bernard Ghanem, and Vladlen Koltun. *Training Graph Neural Networks with 1000 Layers*. ICML 2021. arXiv:2106.07476.
- [2] Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. *DeeperGCN: All You Need to Train Deeper GCNs*. arXiv:2006.07739, 2020.
- [3] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. *Deep Equilibrium Models*. NeurIPS 2019.
- [4] Weihua Hu et al. *Open Graph Benchmark: Datasets for Machine Learning on Graphs*. NeurIPS 2020.